OPTIMIZED CONTEXT RETRIEVAL SYSTEM FOR LLM-BASED CHATBOTS

Comprehensive Project Report

Submitted in Partial Fulfillment of the
Requirements for the Degree of

BACHELOR OF TECHNOLOGY
IN
INFORMATION AND COMMUNICATION TECHNOLOGY

by

Kishan Munjpara
20BIT010

Under the Guidance of

Dr. Nitin Singh Rajput

Assistant Professor,
Department of Information and Communication Technology
Pandit Deendayal Energy University

Department of Information and Communication Technology
School of Technology, Pandit Deendayal Energy University (PDEU)
Gandhinagar, INDIA, 382426

May 2024

Contents

List of Figures

List of Tables

List of Symbols, Abbreviations, and Nomenclature

LLM	Large Language Model
RAG	Retrieval Augmented Generation
LSTM	Long-Short Term Memory
SVM	Support Vector Machine
DPR	Dense Passage Retrieval
AI	Artificial Intelligence
FAISS	Facebook Artificial Intelligence Similarity Search
GloVe	Global Vector
DAN	Deep Averaging Network
ANNS	Approximate Nearest Neighbor Search
OCR	Optical Character Recognition
XG BOOST	Extreme Gradient Boosting
RS	Retrieve Single
RM	Retrieve Multiple
SM	Summary Multiple
SS	Summary Single

1. Introduction

1.1 Introduction of company

International Center of Excellence in Mining (ICEM) is an initiative by the government of Gujarat under the Gujarat Mineral Development Corporation. It aims to develop the growth plan for the mining sector, helping GMDC to become more efficient, safe, and sustainable. It has strategic collaborations with reputed national and international institutions. It focuses on sustainable and environmentally friendly mining also integrating AI and IoT technology to digitize the workflow. Helping GMDC in their management and operation to grow and increase profits.

1.2 Problem Statement

The growing volume of textual data within organizations presents a significant challenge: how to efficiently and accurately retrieve the information needed. This is crucial for various applications like question-answering systems, recommendation systems, and traditional information retrieval. However, conventional search methods often struggle to deliver precise and timely results, leading to wasted time and potential errors.

This thesis proposes a solution by exploring the implementation of Dense Passage Retrieval (DPR) with the Facebook AI Similarity Search (FAISS) library. DPR is a recent advancement in information retrieval that utilizes powerful transformer-based models to encode both documents and queries into high-dimensional vector spaces. Similar documents and queries will have vectors closer together in this space, allowing for highly relevant retrieval. FAISS, for similarity search, efficiently retrieves the most relevant passages from vast document collections based on their vector representations. This combined approach offers significant advantages over traditional keyword-based search, as it considers the semantic meaning of text for more accurate retrieval.

This thesis further delves into Retrieval-Augmented Generation (RAG), This model can then generate summaries, translate retrieved information, or even engage in conversation-like interactions with users. This conversational access to documents can significantly improve information acquisition and comprehension, especially for complex documents. Our research will compare the performance of DPR and RAG for various tasks, highlighting the benefits of RAG, particularly in areas where DPR might have limitations.

2. Literature Review & Problem Identification

2.1 Retrieval Augmented Generation

It has been demonstrated that pre-trained neural language models can pick up a significant amount of in-depth knowledge from data [1]. They may have "hallucinations," struggle to easily enlarge or edit their recollection, and struggle to directly offer insight into their forecasts [2]. Some of these problems can be resolved by hybrid models that integrate parametric memory with non-parametric (i.e., retrieval-based) memories. This is because accessed knowledge can be examined and analyzed, and knowledge can be immediately altered and expanded [3].

Their findings demonstrate the benefits of combining parametric and non-parametric memory when addressing information-intensive tasks that are too complex for humans to complete without the assistance of outside knowledge sources. Their RAG models beat recent approaches using specific pre-training goals and score exceptionally well across various question-answering metrics. Their strategy of unrestrained generation is more effective than earlier methods, although being extractive. Compared to baseline models like BART, their models provide more factual, detailed, and diversified responses in tasks like inquiry generation and fact verification. Regarding fact verification tasks like FEVER, their performance is nearly identical to that of the most advanced pipeline models that use robust retrieval supervision. Finally, they use the replacement of non-parametric memory to update information as the world changes, demonstrating the adaptability of their models [4].

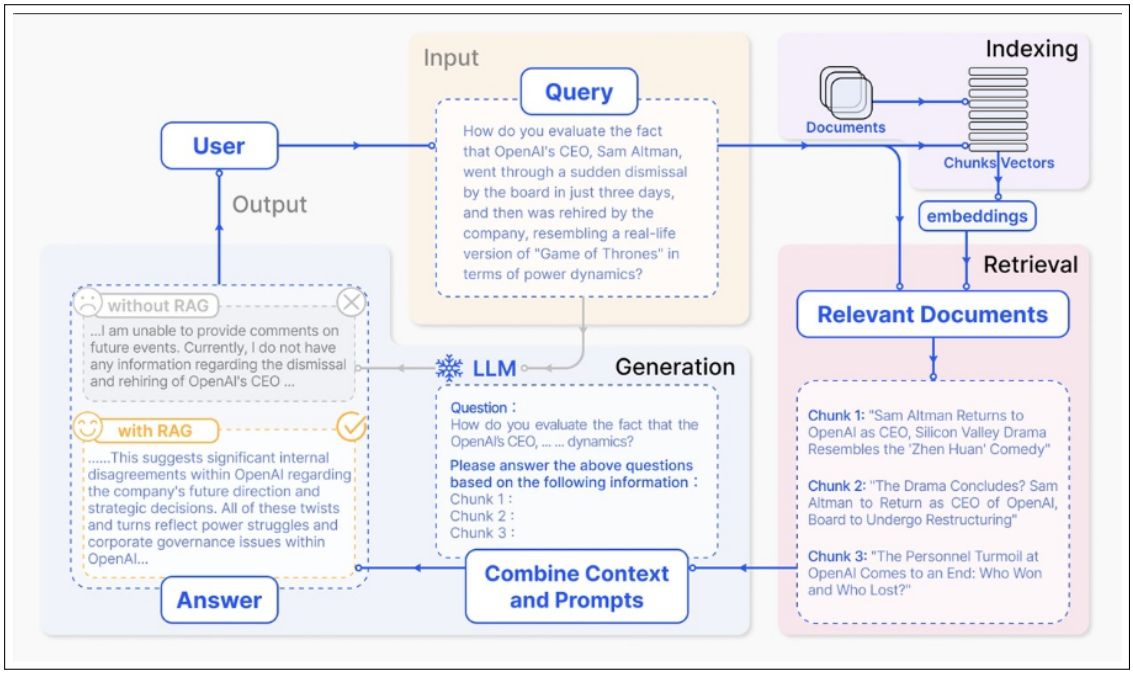


Figure 2.1: RAG Data flow pipeline

RAG is a research paradigm that has been created as a result of ChatGPT which is a widespread application. It follows a characteristic "Retrieve-Read" approach with phases of constitution, retrieval, and indexing.

Data cleanup and file conversion could be the first step, among many others, in indexing. Words, html, pdf, Markdown, etc. The source data is contained in a variety of formats. What to make of language models is getting over that, the sentence is broken into smaller pieces as hashable documents using the RAG Token Model and represented using an embedding model. When these vectors can be recovered, they are saved to a database in a form such that a simple and fast search for similarity conditionals can be carried out.

The RAG-Token version makes it possible to plot a separate latent document for each target token and thereafter multiply keeping in mind the corresponding factors. The organization of this mechanism enables the generator to cherry-pick several different documents and produce a response. The retriever is responsible for retrieving the top K documents according to the query. As a result, the model distributes the next output token for every text of the document. The cycle is repeated and distributions are constitutionally marginalized among all selected groups of documents.

Formally, they define formula as Formally, they define formula as:

$$p_{RAG-Token}(y|x) \approx \prod_i^N \sum_{z \in top-k(p(\cdot|x))} p_\eta(z|x) p_\theta(y_i|x, z, y_{1:i-1})$$

To conclude, another effect RAG has on the sequence recognition duties additionally. This event implies that the selected target class is a one-step forward sequence. Therefore, rationalism-recency and rationalism-tokenization are the same ways.

The RAG-Sequence approach is based on the concept of using an underlying single document with strict word-for-word retrieval. In other simple languages, the probability that (p(y—x) the retrieved document would make a single latent variable will yield a top-sequence through K approximation. Hence, in the example, the top K documents have been retrieved by the retriever and then the generator works out the probability of each document's output sequence. Marginalization of these probability distributions finally results in the sequence probability.

$$p_{RAG-Sequence}(y|x) \approx \sum_{z \in top-k(p(\cdot|x))} p_\eta(z|x) p_\theta(y|x, z) = \sum_{z \in top-k(p(\cdot|x))} p_\eta(z|x) \prod_i^N p_\theta(y_i|x, z, y_{1:i-1})$$

A user's request is converted to vector representation based on the indexing encoding for identification. The K most correlated chunks are those that have more similarities and they are extracted once the vector similarity scores of the given vector and the indexed chunks are computed. The following paragraphs offer you a powerful tool to reinforce or give more details to the prompts.

A broad language model is made to respond meaningfully when it's given either a prompt or a query as well as a reading sentence during the generation process. A strategy upon which the model responds by following the instructions of the assignment may be different, either by using its internal knowledge or by the specified group of documents, which will limit their perception. Conversational history is an integral part of prompts that allows two parties to interact effectively in the dialogue exchanges for conversational use.

Although challenging RAG came up with a lot of obstacles. Imaging precision and memory are the prime issues, and vital parts are often felt neglected and tend to find and select function-less or matching cues. Delusions, lack of theoretical support, and generating improper and irrelevant findings are things that the new generation will deal with.

Connecting the material obtained during the specific task may hinder coherence and cause fragmentation, and the repetition of information due to multiple sources can lead to repetitive answers. Retrieval practice questions and the reply may sometimes carry information that is not related at all and the entire source may be a compilation of redundant information that may be repeated throughout the answers. The issue is that there might been too much faith in that the generation modes work on the additional data, as opposed to the knowledge, which is obtained by written literature or synthesized experience data or any other source.

The paper concludes that RAG's continued success in blending parameterized knowledge from language models with vast non-parameterized external databases is indisputable evidence that RAG is a significant progress toward realizing the potential of Large Language Models (LLMs) to transform the world of language technologies. It is an overview outlining how RAG technologies have developed since their emergence and showing them in several activities. The analysis identifies three developmental paradigms within the RAG framework: Unsophisticated, Sophisticated, and Composable RAG are all versions of the technology that vary in their level of development and therefore in their overall functionality. A highlighted technical aspect of RAG's interlacing with other AI machinery, including fine-tuning and reinforcement learning, has enhanced its effectiveness.

Even if it has made its way on its path, RAG technology still leaves many issues for research to solve, including those related to the strength of the emotional impact and the ability to cover longer contexts. The role of RAG is being extended to embrace multimodal domains for their adaptation to interpret and process different data forms such as images and videos, and even the code. It pinpoints the logic behind the remark by RAG about the implication and application of AI, and that is about the interest triggered by the academy and the industry.

Exhibiting the production of AI applications with RAG as the core as well as an abundant variety of support tools is quite unsubtle a feature of the RAG environment. With the growing volumes of digital technologies in RAG's systems, the need for adjusting the evaluation technique arises for it to conform with the system. Overseeing and adjusting the extent of fairness and matching the field of work with the research and development of AI is a lot of work that steps into the understanding of the impact of RAG on AI researchers and developers.

2.2 Dense Passage Retrieval (DPR)

2.2.1 Idea of Dense Passage Retrieval

For answering factual questions using a large collection of data, is Open-domain Question Answering[1]. Early systems were complicated and consisted of multiple components [2]. As the system became advanced the new method suggested a system as a much simpler 2-stage network framework.

1. Context Reader: A small subset of passages is selected, among which a passage will contain the actual answer.

2. Reader: This will identify the correct passage by reviewing retrieved contexts. [3]

but there exists a large performance deterioration in parts of retrieval.

The most used methodologies for retrieval are **TF-IDF** or **BM25** [4], the logic used by these methods is keyword matching with inverted index, which will demonstrate question and context in higher dimensional sparse vectors. On the contrary, the *dense* encoding, which is latent semantic encoding is complemental to *sparse* representation by design. For example, 2 different equivalent words having 2 different tokens might still be mapped to vectors close together in case of dense encoding. The flexibility of having task-specific representations also can be obtained by leveraging the feature that *dense* encoding are *learnable*. With the help of in-memory data structures and indexing of vectors, Maximum inner product search **MIPS** can be used for efficient retrieval[5].

However, the general belief is there is a need for large amounts of labeled data of questions and contexts to learn good dense vector representation. Thus dense retrieval methods have never outperformed TF-IDF/BM25 for Open-domain Question Answering before ORQA[6], which suggested a sophisticated inverse cloze task (ICT) objective. The proposed method is that the block contains a masked sentence, for additional pertaining.

Using pairs of questions and answers, the reader model and the question encoder are cooperatively tuned during the refinement process. Although ORQA achieves new state-of-the-art performances across multiple open-domain QA datasets demonstrating the superiority of dense retrieval over BM25, it has two key drawbacks. First off, there is a lot of work involved in ICT pretraining, and it is yet unclear how well ordinary sentences work as stand-ins for questions in the objective function. Second, since question-answer pairings aren't used to fine-tune the context encoder, its representations might not be ideal.

This paper explores the possibility of training a superior dense embedding model using only question and passage (or answer) pairs, without additional pretraining. Leveraging the widely used BERT pre-trained model [7] and a dual-encoder architecture [8], they concentrate on devising the appropriate training regimen with a relatively small set of question-passage pairs. Through meticulous ablation studies, they arrive at a remarkably straightforward solution: optimizing the embedding to maximize inner products between question and relevant passage vectors, with an objective function that compares all pairs of questions and passages in a batch. their Dense Passage Retriever (DPR) exhibits exceptional performance. It not only surpasses BM25 by a considerable margin (65.2% vs. 42.9% in Top-5 accuracy) but also significantly enhances end-to-end QA accuracy compared to ORQA (41.5% vs. 33.3%) in the open Natural Questions scenario [9].

The paper makes two contributions. First of all, they show that BM25 may be significantly outperformed by just fine-tuning question and passage encoders on already-existing question-passage pairings with the appropriate training configuration. Furthermore, our empirical results imply that further pretraining might not be required. Second, they confirm that there is a correlation between improved

retrieval precision and increased end-to-end QA accuracy in the context of open-domain question answering. Compared to numerous more complex systems, they get equivalent or better results across several QA datasets in the open-retrieval environment by using a contemporary reader model on the most retrieved sections.

The Dense Passage Retriever (DPR) creates an index for each of the M passages that are meant to be retrieved by mapping any text passage to a d-dimensional real-valued vector using a dense encoder, represented as EP($\cdot$). DPR uses a unique encoder, EQ($\cdot$), at runtime to translate the input question into a d-dimensional vector. Then, k passages with vectors that are most similar to the question vector are retrieved. The dot product of the vectors corresponding to the question and the passage is used to determine how similar they are:

$$sim(q, p) = E_Q(q)^t E_P(p)$$

Although more intricate model architectures, including networks with several layers of cross-attention, can be used to measure the similarity between a passage and a question, it is essential that the similarity function be decomposable in order to enable passage representations to be precomputed. The majority of decomposable similarity functions entail Euclidean distance (L2) transformations. For instance, the Mahalanobis distance is equal to the L2 distance in a converted space, and cosine similarity is equal to the inner product for unit vectors. The use of inner product search and its relationship to cosine similarity and L2 distance have been studied in great detail.[10]. As indicated by our ablation study, where other similarity functions perform similarly (Section 5.2; Appendix B) of the paper, they opt for the simpler inner product function to enhance the dense passage retriever by refining the encoders.

In this study, although the question and passage encoders could theoretically be constructed using any neural network architecture, two separate BERT [7] networks (base, uncased) are employed. The output is obtained by considering the representation at the [CLS] token, resulting in a dimensionality of d = 768.

2.2.2 Encoder

For numerous NLP tasks, the scarcity of training data poses a significant hurdle, challenging data-intensive deep-learning approaches. Due to the exorbitant expense associated with annotating supervised training data, extensive training sets are typically unavailable for most NLP tasks in both research and industry settings. Many models tackle this issue by implicitly engaging in limited transfer learning, leveraging pre-trained word embeddings such as those generated by word2vec [11] or GloVe [12]. However, recent research has showcased impressive transfer task performance by utilizing pre-trained sentence-level embeddings [13].

the model architecture for the two encoding models is discussed. The two encoders are designed with distinct goals in mind. One, based on the transformer architecture, prioritizes high accuracy despite higher model complexity and resource consumption. The other encoder focuses on efficient inference, albeit with a slight reduction in accuracy.

Transformers:

As described in [14], the sentence encoding model based on the transformer architecture uses the encoding sub-graph to create sentence embeddings. Using attention mechanisms, this sub-graph creates context-aware representations of words in a phrase by taking into account the identity and order of all other words. The element-wise sum of the representations at each word position is then calculated to combine these context-aware word representations into a fixed-length sentence encoding vector. The encoder creates a 512-dimensional vector as the sentence embedding from a lowercase PTB tokenized string as input. The encoding model, which is intended for general-purpose use, uses multi-task learning to allow one model to handle several downstream jobs. These challenges include a conversational input-response task [15] for incorporating parsed conversational data, a SkipThought-like task [16] for unsupervised learning from arbitrary running text, and several classification tasks for supervised learning. A transformer-based model takes the role of the original LSTM model [17] in the Skip-Thought task. The transformer-based encoder delivers the highest overall transfer task performance, as shown by the experimental findings. Sentence length has a major impact on compute time and memory usage, which are raised with this modification.

Deep Averaging Network (DAN)

According to [16], a deep averaging network (DAN) is used in the second encoding model. In this model, to generate sentence embeddings, input embeddings for words and bi-grams are first averaged together and subsequently fed into a feedforward deep neural network (DNN). Similar to the Transformer encoder, the DAN encoder generates a 512-dimensional phrase embedding using a lowercase PTB tokenized string as input. The Transformer-based encoder and the DAN encoder are trained using a similar method called multi-task learning, in which a single DAN encoder is used for several downstream jobs. The DAN encoder's linear compute time as a function of input sequence length is one of its main benefits. Similar to the findings by [16], their results indicate that DANs achieve strong baseline performance on text classification tasks.

While any neural network architecture may theoretically be used to build the question and passage encoders, two distinct BERT [7] networks (base, uncased) are used in this DPR study. Taking into account the representation at the [CLS] token yields an output with a dimensionality of d = 768.

2.2.3 Inference

The emergence of deep learning has led to a revolution in the way complicated data is stored and retrieved, most notably with the creation of embeddings. These embeddings function as intermediate representations for further processing, e.g., SimCLR uses pretext tasks for self-supervision, or self-supervised image embeddings are used as input for shallow supervised image classifiers [18]. Our paper centers on the direct application of embeddings in media item comparisons. The embedding extractor is designed in a way that makes sure the distance between embeddings corresponds to how similar the relevant media items are to each other. As a result, doing a similarity search between media items is made simple by performing a neighborhood search in this vector space.

In industrial environments, embeddings are frequently used for activities where

end-to-end learning would be ineffective. For example, it is more practical to upgrade a k-nearest-neighbor classifier than a deep neural net for classification. In these situations, embeddings function as a versatile, compact intermediate representation. This explains why industrial database management systems (DBMS) with vector storage and search capabilities are becoming more and more popular. These database management systems (DBMSs) sit at the nexus of approximate nearest neighbor search (ANNS) algorithms and conventional databases; ANNS was previously limited to particular use cases or research.

In practical applications, there are several benefits to keeping the functions of the vector search method and embedding extraction distinct. Regarding the embedding distance, both are constrained by a "embedding distance":

- The embedding extractor is trained to align embedding distances with the job at hand; in current systems, this is often a neural network.
- Given the accepted distance metric, the vector index seeks to as correctly conduct a neighbor search among the embedding vectors.

Faiss is an ANNS library of industrial grade that may be used in basic scripts or as part of a database management system. Faiss is a toolbox that includes a variety of indexing techniques, including preprocessing, compression, non-exhaustive search, and more, in contrast to other libraries that concentrate on just one indexing technique. This adaptability is essential since different usage scenarios call for different indexing strategies to be most effective.

It's important to clarify what Faiss is not:

- Faiss only indexes embeddings that have been extracted via an alternative approach; it does not extract features.
- Faiss provides functions that are executed as a part of the local machine's calling process; it is not a service.
- Faiss is not a database; it does not provide consistency, load balancing, sharding, or concurrent write access. The library purposefully narrows its scope to concentrate on well thought out algorithms.

The index, which is the central component of Faiss, can hold many database vectors that are added to it piecemeal. A query vector is sent to the index during the search process, and it returns the database vector that is closest to the query vector in terms of Euclidean distance. There are several variations of this feature, such as finding the k-nearest neighbors back, searching in a given range, searching in batch mode, supporting metrics other than Euclidean distance, and sacrificing search accuracy in favor of speed or memory usage. Both CPUs and GPUs can be used for search activities.

All passages are processed by the passage encoder E_P during inference, and FAISS is used to index them offline [**?**]. FAISS is an incredibly effective open-source toolkit that focuses on dense vector clustering and similarity search. It can easily handle billions of vectors. The top k passages with embeddings closest to v_q are obtained when a question q is provided at runtime, after which its embedding $v_q = E_Q(q)$ is calculated.

2.2.4 DPR Training

Metric learning presents a hurdle when training the encoders to maximize the dot-product similarity (Eq. (1)) as a reliable ranking function for retrieval [19]. By learning an enhanced embedding function, the goal is to create a vector space where pairs of questions and passages that are relevant have fewer distances (i.e., higher similarity) than irrelevant ones.

Let $\mathcal{D} = \{\langle q_i, p_i^+, p_{i,1}^-, \ldots, p_{i,n}^- \rangle\}_{i=1}^m$ represent the training data comprising m instances. Each instance contains a question q_i and one relevant (positive) passage p_i^+, along with n irrelevant (negative) passages $p_{i,j}^-$. The loss function is optimized as the negative log-likelihood of the positive passage:

$$L(q_i, p_i^+, p_{i,1}^-, \ldots, p_{i,n}^-) = -log \frac{e^{sim(q_i, p_i^+)}}{e^{sim(q_i, p_i^+)} + \sum_{j=1}^n e^{sim(q_i, p_{i,j}^-)}}$$

Positive examples are frequently provided explicitly for retrieval tasks, but negative examples need to be chosen from a large pool. For example, a QA dataset may contain passages that are pertinent to a question, or the answer may be used to identify the text. Unless otherwise noted, every other passage in the collection can be regarded as unimportant by default. Although it is frequently disregarded in practice, the choice of negative samples can have a big influence on the encoder's quality. Three types of negative examples are considered:

1. Random: any randomly chosen passage from the corpus.

2. BM25: top passages returned by BM25 that do not contain the answer but match most question tokens.

3. Gold: positive passages paired with other questions appearing in the training set.

In Section 5.2 of the paper of DPR, the impact of various negative passage kinds and training methods is covered. One BM25 negative passage and gold passages from the same mini-batch are used in the best model. In particular, significant performance benefits can be obtained while processing can be streamlined by employing gold passages from the same batch as negatives. The details of this strategy are provided below.

Regarding in-batch negatives, let us examine a mini-batch consisting of B questions, each corresponding to a pertinent portion. The question and passage embeddings in the batch of size B are represented by the $(B \times d)$ matrices, denoted Q and P. A $(B \times B)$ matrix of similarity scores is formed by $S = QP^T$, where each row represents a question linked with a B passage. This method allows for efficient training on B^2 (q_i, p_j) question/passage pairs in each batch because computation is reused. Any (q_i, p_j) pair in this configuration is considered negative if $i = j$ and positive otherwise. Each batch thus creates B training examples, where every question has $B - 1$ negative passage.

The technique of in-batch negatives has been previously employed in the full batch setting [20] and more recently adapted for mini-batches[15] [21]. It has demonstrated effectiveness in learning dual-encoder models by increasing the number of available training examples.

Dataset Used:

Dataset	Train		Dev	Test
Natural Questions	79,168	58,880	8,757	3,610
TriviaQA	78,785	60,413	8,837	11,313
WebQuestions	3,417	2,474	361	2,032
CuratedTREC	1,353	1,125	133	694
SQuAD	78,713	70,096	8,886	10,570

Figure 2.2: Used datasets for training DPR.

2.2.5 DPR Results

The percentage of the top 20 out of 100 recovered passages that include the answer is known as the Top-20 & Top-100 retrieval accuracy on test sets. Single and Multi indicate whether individual or combined training datasets (all the datasets excluding SQuAD) were used to train our Dense Passage Retriever (DPR).

Training	Retriever	Top-20					Top-100				
		NQ	TriviaQA	WQ	TREC	SQuAD	NQ	TriviaQA	WQ	TREC	SQuAD
None	BM25	59.1	66.9	55.0	70.9	68.8	73.7	76.7	71.1	84.1	80.0
Single	DPR	78.4	79.4	73.2	79.8	63.2	85.4	**85.0**	81.4	89.1	77.2
	BM25 + DPR	76.6	79.8	71.0	85.2	**71.5**	83.8	84.5	80.5	92.7	**81.3**
Multi	DPR	**79.4**	78.8	**75.0**	**89.1**	51.6	**86.0**	84.7	**82.9**	93.9	67.6
	BM25 + DPR	78.0	**79.9**	74.7	88.5	66.2	83.9	84.4	82.3	**94.1**	78.6

Figure 2.3: Comparison of Test Results of DPR.

Using varying amounts of training instances, retriever top-k accuracy was compared to BM25 in dense passage retrieval. The development set of Natural Questions is used to gauge the outcomes. Already, DPR trained with 1,000 instances works better than BM25.

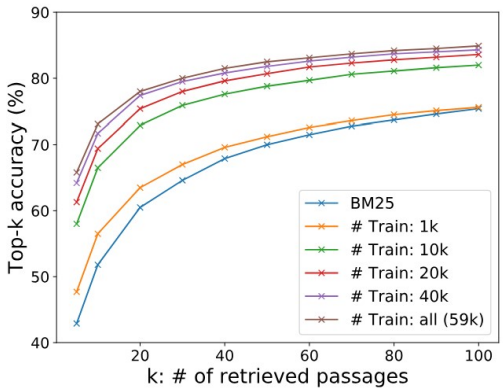


Figure 2.4: Comparison of Accuracy with number of passages retrieved curve.

2.3 Knowledge Guided Text Retrieval and Reading for Open Domain Question Answering

This paper primarily focuses on providing a new approach for text retrieval and reading in Open Domain Question Answering (QA). Open domain QA aims

to answer user queries accurately by retrieving and analysing text information from text sources. The problem with the traditional QA systems is using text matching for passage retrieval, ultimately leads to limited coverage and accuracy. To overcome these problems recent work focused on including external knowledge sources to improve retrieval and reading processes.

This approach retrieves and reads the passage graph. Graph represented with vertices and edges, where vertices represent text information and edges represent the relationships. To derive relationships an external knowledge base or co-occurrence in the same article. This method can increase coverage by using knowledge-guided retrieval to find more relevant information.

This method improved performance by 2-11% over a non-graph baseline testing using three open domain datasets, WEB QUESTIONS, NATURAL QUESTIONS and TRIVIAQA. This approach gives improved results without using costly model training.

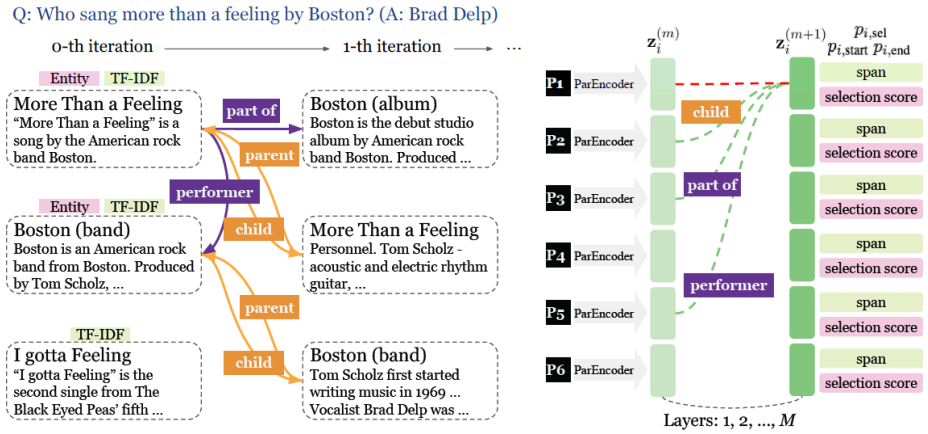


Figure 2.5: Diagram of GRAPHRETRIEVER and GRAPHREADER proposed approach

This new general approach consists of two models retrival mode GRAPHRETRIEVER and neural reader model GRAPHREADER. The proposed overall approach is shown in the above graph. GRAPHRETRIEVER retrieves the group of passages denoted with vertices and edges representing passage and their relationships between passages. This integrated approach gave a significant advancement in the open domain QA field offering more coverage and precise information retrieving.

2.3.1 GRAPH RETRIEVER

It takes a question as input and uses the knowledge base to construct a graph. It starts with a set of articles extracted using corresponding question entities. The top K articles will be extracted using a TF-IDF similarity search. The first passage was chosen as seed passage P(0).

P(0) will be used as a starting point. Now retriever expands the passage graph from P(m-1) to P(m) by iterations. First, the graph will be updated by appending passages that are related to P(m-1) based on the relation in data. So p_j is added

through the relation $r_{i,j}$. Second, supporting passages are also appended to the graph. Non-first passages articles that are related to P(m-1) and it is ranked by BM25. Top k passages will be chosen to construct relations. Finally, a passage graph consisting of n passages: $p_1, p_2, .., p_n$. Where relation between passage pair (p_i, p_j) denoted by $\{ r_{i,j} | 1 \leq i, j \leq n \}$, $r_{i,j}$ is child, parent or no relation.

2.3.2 GRAPH READER

The reader takes input question q and n retrieved passages $p_1, p_2, .., p_n$ and outputs an answer. This approach gives knowledge-rich representation by doing a fusion of graph structure. Graph reader obtains a question-aware passage representation.

$$P_i = TextEncode(q, p_i) \in \mathbb{R}^{L \times h}$$

where q is the question, passage P_i, L is the maximum length of each passage and h is the hidden dimension. For this BERT encoder is used to encode but different encoders can be applied. The reader also encodes a relation $r_{i,j}$ using a relation encoder.

$$\mathbf{r}_{i,j} = RelationEncode(r_{i,j}) \in \mathbb{R}^h$$

It builds M graph-aware fusion layers to update passage representation by circulating information through the edges of the graph. It initialises representation with max pooling giving new representation $z_i^{(m)}$ for each fusion layer $1 \leq m \leq M$. There are two methods to obtain representation. The binary method gives whether this passage pair is related or not without incorporating relations. Relation-aware uses a relation-aware composition function.

$$z_i^{(m+1)} = \sum_{j=1}^{n} (W_f [z_i^{(m)} \oplus f(r_{i,j}, z_j^{(m)})] + b_f)$$

where f is the composition function, $\oplus$ is a concatenation. learnable parameters are W_f and b_f.

GRAPHREADER uses this updated passage representation $z_1^{(m)}, z_2^{(m)}, ..., z_n^{(m)}$ to compute the probability of p_i to be related. For training, the maximum marginal likelihood objective is to maximize.

$$\sum_{i=1}^{n} \mathbb{I}[|\mathcal{S}_i| > 0] \log P_{sel}(i) + \sum_{i=1}^{n} \log(\sum_{(s_i,e_i) \in \mathcal{S}_i} (P_{start,i}(s_i) \times P_{end,i}(e_i))$$

They hypothesize span prediction are accurate and easy task rather than choosing the right evidence passage.

Table 2.1: Overall results on the development and the test set of three datasets.

Retriever	Reader	WebQuestions		Natural Questions		TriviaQA	
		Dev	Test	Dev	Test	Dev	Test
Text-match	ParReader	23.6	25.2	26.1	25.8	52.1	52.1
Text-match	ParReader++	19.9	20.8	28.9	28.7	54.5	54.0
GraphRetriever	ParReader	33.2	33.0	30.2	29.3	54.8	54.7
GraphRetriever	ParReader++	33.7	31.8	33.1	33.5	55.5	55.0
GraphRetriever	GraphReader (binary)	34.0	**36.4**	34.2	34.1	55.2	54.2
GraphRetriever	GraphReader (relation)	34.0	36.0	**34.7**	**34.5**	**55.8**	**56.0**
Previous best (pipeline)		-	18.5[a]	31.7[b]	32.6[b]	50.7[c]	50.9[c]
Previous best (end-to-end)		**38.5**[d]	**36.4**[d]	31.3[d]	33.3[d]	45.1[d]	45.0[d]

In the results, GRAPHRETRIEVER with 1%-11% gains over text matching retrieval and GRAPHREADER with 1%-5% gains this led to fusing information is more effective than reading passages in isolation. GRAPHRETRIEVER and GRAPHREADER perform better than previous graph-based retrieval with a significant margin. An additional pretraining strategy can used to train BERT encoders but not discussed in this paper it has the potential to be trained end to end.[22]

2.4 Large Language Models

Language processing is about to choose an important shift because of Large Language Models (LLM) APIs. With the help of device getting-to-know and deep mastering algorithms, LLM APIs provide by no means-before-visible access to natural language knowledge powers. Developers can now layout applications that may understand and react to written language in methods never visible earlier than via utilizing these new APIs.

The first-rate LLM APIs to be had nowadays, together with Bard, ChatGPT, PaLM, and others, are in comparison here. We'll additionally communicate approximately the effects of language processing and a few use cases for integrating these LLMs.

2.4.1 What Are LLMs?

Large language models (LLMs) are Artificial intelligence models made to process, comprehend, and produce natural language. These models primarily rely on deep learning algorithms that can read, synthesize, and grasp language at a level that is comparable to human ability through the examination of vast amounts of text. Google's BERT (Bidirectional Encoder Representations from Transformers) and OpenAI's GPT (Generative Pre-trained Transformer) are the two most well-known LLMs. Compared to typical natural language processing (NLP) methods, which frequently use manually created rules to analyze and understand text, LLMs are very different. Rather, LLMs are made to analyze vast volumes of text in order to identify and comprehend linguistic patterns. To comprehend word usage patterns, they build an internal representation of language using neural networks.

2.4.2 Top Large Language Model (LLM) APIs

Many businesses and organizations have been putting a lot of effort into developing large, reliable language models as natural language processing (NLP) grows more sophisticated and in demand. These are some of the top LLMs available right now. Unless otherwise specified, all offer API access.

1. ChatGPT

A wonderful example of this is ChatGPT. Chatbots represent one of the most fascinating uses of LLMs. ChatGPT can have natural language conversations with users since it is powered by the GPT-4 language model. Due to its broad training, ChatGPT can help with a wide range of activities, provide answers to inquiries, and hold lively discussions on a wide range of topics. Through the ChatGPT API, you can easily write an email, produce Python code, and even

adjust to various contexts and conversational styles. The models' underlying API is accessible through ChatGPT's parent business, OpenAI. This OpenAI Chat Completions API call serves as an example.

Listing 2.1: OpenAI Chat Completions API call

```
import openai

openai.ChatCompletion.create(
  model="gpt-3.5-turbo",
  messages=[
        {"role": "system", "content": "You are a helpful
           assistant."},
        {"role": "user", "content": "Who won the world
           series in 2020?"},
        {"role": "assistant", "content": "The Los
           Angeles Dodgers won the World Series in 2020.
           "},
        {"role": "user", "content": "Where was it played
           ?"}
    ]
```

The example response:

Listing 2.2: OpenAI Chat Completions API call response

```
{
 'id': 'chatcmpl-6p9XYPYSTTRi0xEviKjjilqrWU2Ve',
 'object': 'chat.completion',
 'created': 1677649420,
 'model': 'gpt-3.5-turbo',
 'usage': {'prompt_tokens': 56, 'completion_tokens': 31,
     'total_tokens': 87},
 'choices': [
   {
    'message': {
      'role': 'assistant',
      'content': 'The 2020 World Series was played in
         Arlington, Texas at the Globe Life Field, which
         was the new home stadium for the Texas Rangers
         .'},
    'finish_reason': 'stop',
    'index': 0
   }
  ]
}
```

2. Bard

Google created Bard, an AI chatbot that mimics human speech and visuals using its Large Language Model (LLM) and Language Model for Dialogue Applications (LaMDA). Bard is conversational, so users may write a question and get a personalized response in natural language, unlike Google Search. A compelling illustration of how LLMs can be utilized to produce potent conversational AI experiences is Bard. Based on input from a particular user, the system can produce text and graphics naturally and interestingly. The API is request-only at the moment. Thus, you have to ask for access to implement it.

3. GooseAI

GooseAI is another useful LLM API on the market. GooseAI is a fully managed NLP-as-a-Service that is provided through an API. It provides an unmatched speed and a cutting-edge range of GPT-based language models. GooseAI also provides further customization and language model possibilities. Users can select from a variety of GPT models and modify them to meet their unique requirements. The Goose.AI API was made to be compatible with other similar APIs, such as OpenAI.

4. Cohere

Cohere is another player in the field of huge language models. With the use of cutting-edge natural language processing (NLP) technology, this innovative solution enables developers and businesses to create amazing products while protecting the privacy and security of their data. Businesses of all sizes can create, explore, and search for information in novel ways using Cohere. Because the models have already been pre-trained on billions of words, the API is simple to use and adapt. This implies that even smaller companies can now benefit from this potent technology without having to break the bank.

5. Claude

Based on Anthropic's research, Claude is a next-generation AI helper that highlights the potential of LLM APIs. With Claude, developers may use the potential of big language models by gaining access to an API and chat interface via the developer console. There are numerous applications for Claude, such as search, Q&A, coding, creative and collaborative writing, summarization, and more. Early adopters have noted that Claude is more steerable, easier to communicate with, and less likely to generate detrimental outputs than other language models available on the market.

6. LLaMA

An interesting model that is worth bringing up in the discussion of LLMs is called Language Learning and Multimodal Analytics, or LLaMA. LLaMA was created by the Meta AI team to explicitly tackle the problem of language modeling with minimal processing capacity. In the large language model domain, LLaMA is especially appealing since it needs less processing power and resources to test novel ideas, validate existing research, and investigate new use cases. It does this by adopting a novel strategy for model inference and training, utilizing transfer learning to create new models more quickly and with fewer resources. As of this

writing, the API only accepts requests.

7. PaLM

With an efficient model in terms of both size and capabilities, Google's Pathways Language Model (PaLM) API offers a simple and secure approach to build on top of language models. Even better, Pathways AI's MakerSuite offering includes PaLM as only one component. This user-friendly tool is ideal for rapidly prototyping concepts and will soon be enhanced with prompt engineering, the ability to generate synthetic data, and the ability to customize models.

2.4.3 How Is Each LLM Unique?

The fact that every LLM is different and unique from the others is among its most amazing features. These models all have advantages and disadvantages of their own. The LLMs mentioned above are contrasted with one another as follows:

- Bard: Anyone creating interesting content can use this model, which was created especially for creative writing and storytelling.
- ChatGPT: This model is intended for conversational AI and chatbots. Because of its exceptional responsiveness, it can keep up with talks that move quickly while maintaining context.
- GooseAI: This model is ideal for marketers and content creators as it concentrates on producing captivating, high-quality content. Its capacity to detect human emotions and react appropriately sets it apart and makes it extremely desirable.
- Coherence: This model can be used for sentiment analysis, summarization, and text classification, among other NLP applications. It is quite adaptable and may be tailored to meet particular requirements.
- Although this model is a recent addition to the industry, it has already attracted a lot of attention because of its capacity to produce really interesting and unique material. For marketers who want to stand out in a crowded market, it's ideal.
- Built on top of OpenAI's GPT-3 platform, the Azure OpenAI Service is perfect for companies wishing to incorporate language processing into their current systems.
- LLaMA: This methodology is intended to offer tailored suggestions for media, including books, films, and other types of content. It makes recommendations that are specific to the interests of the user and is incredibly accurate thanks to sophisticated algorithms.
- PaLM: This model is intended for language comprehension and can be applied to a variety of natural language processing (NLP) applications, such as search engines, chatbots, and language translators.

2.4.4 Applications of Large Language Models in Real World

Large AI models that have been trained on enormous datasets have proven to be incredibly powerful in a variety of real-world applications. LLMs are revolutionizing communication between individuals and businesses by simplifying interactions with ever-increasing and complicated volumes of data. The following are a few examples of how LLMs are changing the world:

- Content creation: LLMs can help with social media posts, marketing content, and even creative writing.
- Natural language generation and understanding: By enabling chatbots and virtual assistants to comprehend human language and provide pertinent responses, LLMs can increase the usefulness of these technologies for both customers and enterprises.
- Sentiment analysis: To uncover patterns and sentiments that might guide business choices and enhance marketing tactics, LLMs can examine market research or social media posts.
- Machine translation: By mechanically translating text between languages with a high degree of accuracy, LLMs can eliminate language barriers.
- Summarization: By offering a condensed and palatable version of a vast amount of information, LLMs can summarise articles, reports, or other written documents, saving consumers time and effort.
- Personalized language instruction: By tailoring the material to each learner's need, LLMs can help with tutoring and language acquisition.
- Question-answering systems: By providing answers to commonly asked questions, LLMs can help with knowledge bases and customer assistance.
- Text classification: LLMs are capable of classifying text for purposes including document organization, subject classification, and spam filtering.
- Software development and code generation: LLMs can help with software development by writing code or offering recommendations for better code.
- Services for speech recognition and transcription: LLMs can help with speech recognition and accurately transcribe audio.
- Document analysis for medical, legal, and technological fields: LLMs can help professionals by deciphering and condensing complicated documents.
- Tools for accessibility: LLMs can assist those with disabilities by offering speech-to-text or text-to-speech translation.

2.4.5 GPT-4 Vision Feature

The GPT-4 Turbo with Vision model enables the model to recognize photos and provide information about them. In the past, language model systems were restricted to processing text as their only input modality. This limited the domains in which models such as GPT-4 could be applied for a large number of application cases. The model was previously referred to in the API as GPT-4V or gpt-4-vision-preview. Still, the picture inputs are not yet supported by the Assistants API.

There are two basic ways to provide images to the model: either provide the base64 encoded picture directly in the request or provide a URL to the image. Images may be sent via system, assistant, and user messages. Images in the initial system message are not supported at this time. However, this could change in the future.

Listing 2.3: OpenAI Chat Completions API call

```
from openai import OpenAI

client = OpenAI()

response = client.chat.completions.create(
  model="gpt-4-turbo",
  messages=[
    {
      "role": "user",
      "content": [
        {"type": "text", "text": "Whats in this image
           ?"},
        {
          "type": "image_url",
          "image_url": {
            "url": "https://upload.wikimedia.org/
                wikipedia/commons/thumb/d/dd/Gfp-
                wisconsin-madison-the-nature-boardwalk.
                jpg/2560px-Gfp-wisconsin-madison-the-
                nature-boardwalk.jpg",
          },
        },
      ],
    }
  ],
  max_tokens=300,
)

print(response.choices[0])
```

When it comes to providing broad answers regarding the contents of the photos,

the model excels. Although it can recognize the relationships between things in pictures, it is not yet ready to provide precise answers to queries regarding the locations of specific objects. You can ask it questions like what color a car is or what kind of meal to make based on what's in your fridge, but if you show it a picture of a room and ask it where the chair is, it might not respond appropriately.

2.4.6 Limitations of GPT-4 Vision

Although the GPT-4 with vision is strong and useful in a variety of scenarios, it's critical to recognize the model's limitations. We are aware of the following constraints, to name a few:

- Non-English: When handling photos with text written in non-Latin alphabets, such as Japanese or Korean, the model might not function at its best.
- Small text: To make the text easier to read, enlarge it within the image, but don't cut off any crucial features.
- Rotation: Text or images that are upside-down or rotated may be misinterpreted by the model.
- Visual components: Text or graphs with varying colors or line patterns, such as dashed, dotted, or solid, may be difficult for the model to fully understand.
- Spatial reasoning: The model has trouble with activities that need accurate spatial localization, like chess position identification.
- Accuracy: In some cases, the model may produce inaccurate captions or descriptions.
- Image shape: The model has trouble with fisheye and panoramic photos.
- Metadata and resizing: Before analysis, pictures are resized, which modifies their original dimensions. The model does not handle the original file names or metadata.
- Counting: May provide ballpark figures for items in pictures.
- CAPTCHAS: For safety reasons, we've put in place a system to prevent the submission of CAPTCHAs.

2.4.7 Calculating costs

Similar to text inputs, image inputs are metered and charged in tokens. Two things affect an image's token cost: its size and the detail option on each image-url block. Each of the detailed photos costs just 85 tokens. Detail: To preserve their aspect ratio, high photos are first resized to fit inside a 2048 by 2048 square. Next, they are resized so that the image's longest side measures 768 pixels. Lastly, we tally the number of 512 px squares in the picture. Those squares cost 170 tokens apiece. The final total always includes an additional 85 tokens.

Here are a few illustrations demonstrating the above.

- In detail, a 1024 by 1024 square image costs 765 tokens in high mode.

- Since 1024 is smaller than 2048, there isn't a first resize.
- Since 1024 is the shortest side, the image is resized to 768 × 768.
- Since the image requires 4, 512 px square tiles, the total token cost comes to 170 * 4 + 85 = 765.

- Details of a 2048 x 4096 image: high mode costs 1105 tokens.
  - To fit the image inside the 2048 square, we reduce its size to 1024 x 2048.
  - We reduce the size even further to 768 x 1536 because the shortest side is 1024.
  - Since six 512-pixel tiles are required, 170 * 6 + 85 = 1105 is the total token cost.

- A detailed 4096 × 8192 image costs 85 tokens at the lowest.
  - Low-detail photos have a set cost regardless of the size of the input.

2.5 Llama 2: Open foundation and fine-tuned chat models

Llama is a family of closed source models and a suitable substitute of open-source chat models. Llama2-chat are scaled in range from 7 billion to 70 billion and optimized for discourse usage events. Self-supervised extensive corpus is used for pretrain Auto-regressive transformers. Llama-2 outperformed, in case of helpfulness and safety benchmark, all pretrained open-source and closed-source LLMs like MPT 7b.[23].

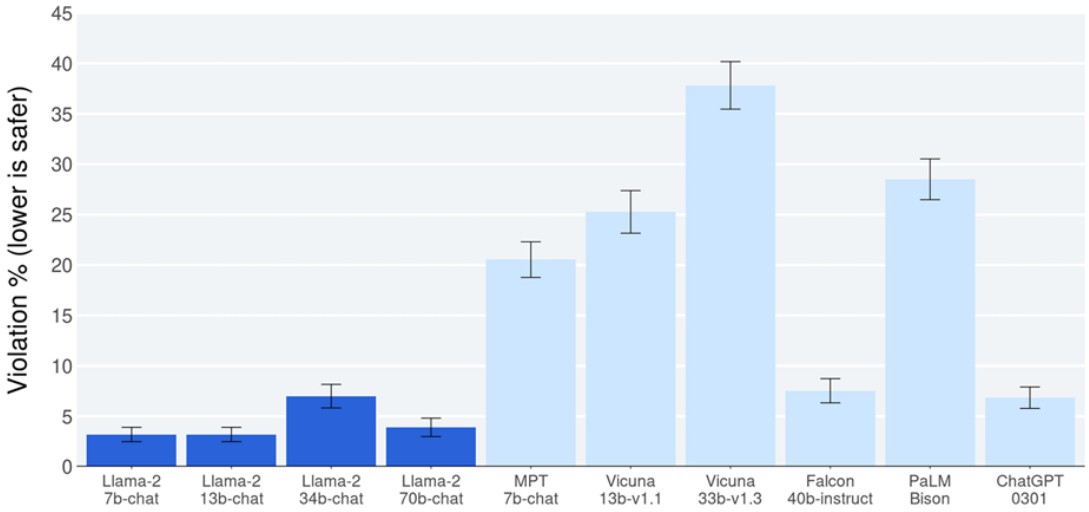


Figure 2.6: Comaparative analysis of violation on different LLM models

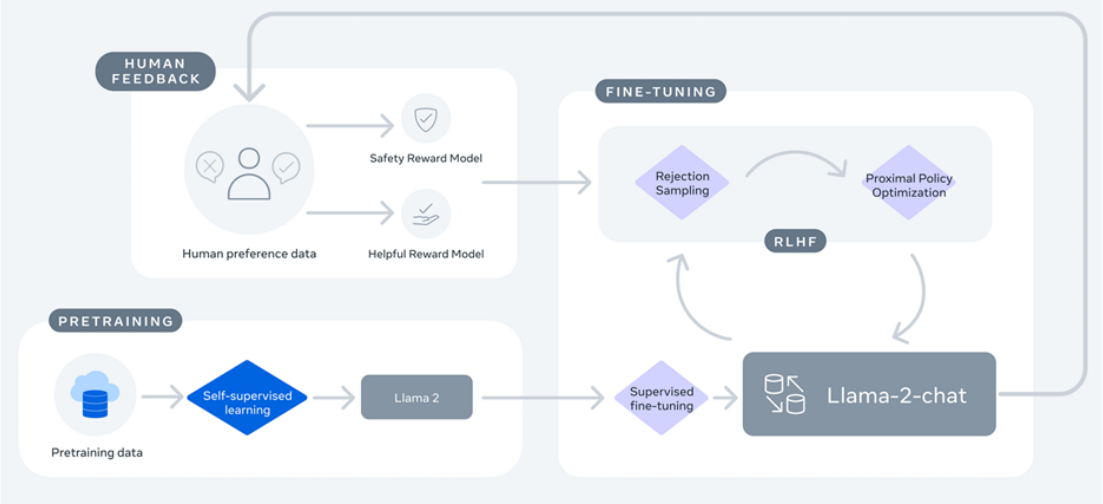


Figure 2.7: LLAMA2 RAG pipline

Initially, Llama-2 was trained using data that was available to the public to teach it fundamental ideas. Subsequently, supervised fine-tuning was used to further enhance Llama-2's learning and provide a preliminary model. Then, using techniques from Reinforcement Learning with Human Feedback (RLHF) and Proximal Policy Optimization (PPO) for rejection sampling, the model was improved iteratively.

To achieve this goal, we repeatedly update the policy by sampling prompts P from our dataset D and generating g from the policy π. We do this by using the PPO algorithm and loss function.

$$R(g \mid p) = \tilde{R}_c(g \mid p) - \beta D_{KL}(\pi_\theta(g \mid p) \parallel \pi_0(g \mid p))$$

$$R_c(g \mid p) = \begin{cases} R_s(g \mid p) & \text{if IS\_SAFETY}(p) \text{ or } R_s(g \mid p) < 0.15 \\ R_h(g \mid p) & \text{otherwise} \end{cases}$$
$$\tilde{R}_c(g \mid p) = \text{WHITEN}(\text{LOGIT}(R_c(g \mid p)))$$

The gathered paired human preference data is converted into a binary ranking label format to train the reward model. This arrangement guarantees that the selected answer receives a greater mark than its equivalent. A binary ranking loss with a margin component is used in the training process to help effectively separate replies into preferred and non-preferred categories based on human feedback.

$$\mathcal{L}_{\text{ranking}} = -\log(\sigma(r_\theta(x, y_c) - r_\theta(x, y_r) - m(r)))$$

Throughout the entire procedure, the focus on acquiring reward modelling data was maintained to guarantee that the model's profits matched the expected distributions. In conclusion, A new family of pre-trained and enhanced models, Llama 2, with parameters ranging from 7 billion to 70 billion. The researchers

discovered that these models perform similarly to some of the proprietary models they assessed and are comparable to open-source chat models that are currently in use. They did, however, admit that in terms of performance, they are still trailing behind more sophisticated versions like GPT-4. With a focus on safety and utility, the researchers gave a thorough explanation of the techniques and strategies used to create these models. To further advance research and make a greater contribution to society, the researchers made both Llama 2 and Llama 2-Chat public. They also declared intentions to enhance llama, indicating their unwavering dedication. [23]

3. Methodology

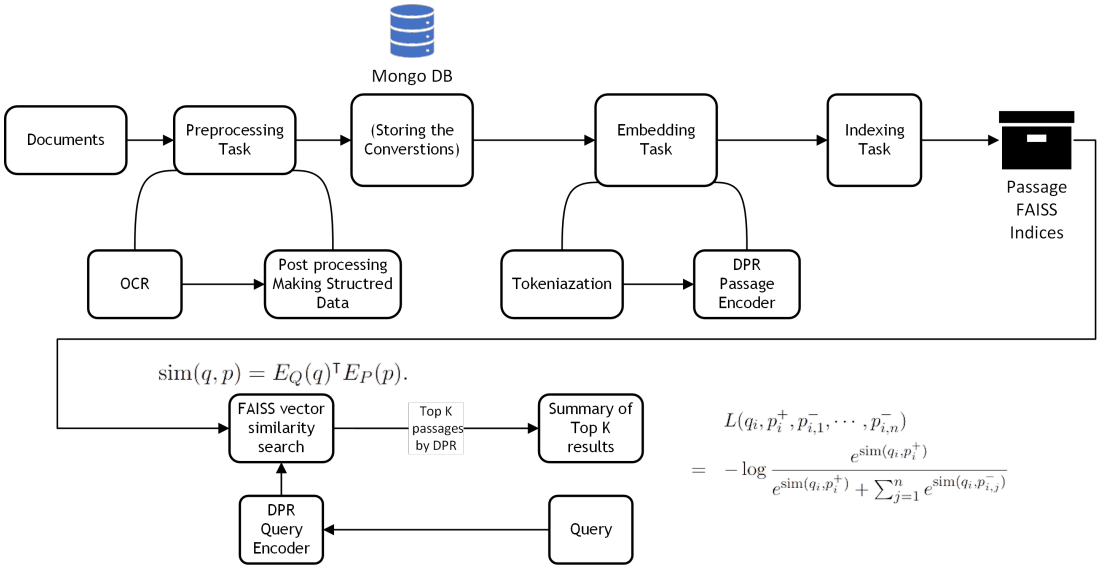


Figure 3.1: DPR system pipline

3.1 Data Preprocessing

Data acquisition

This system uses meeting transcripts as a data source for retrieval. As per the law of confidentiality, it is not allowed to show other data files without consent, Meeting transcripts are used as data sources for the primary prototype stage of this system, as well as for testing and validating. Total 8 meeting transcript PDFs of quarterly meetings between the years 2021 and 2023. Average of 17 pages for transcripts.

OCR and Data Transformation

Optical Character Recognition (OCR) is one of the crucial steps in a preprocessing task. The information must be correct data should not be lost during OCR. Meeting transcripts are primarily structured in speaker and content format shown in Fig 3.2. This allows better OCR and also hard-to-interpret structure to OCR tool for extracting structured text format same as written in meeting transcripts.

PyMUPDF library of Python gives the best result in OCR. Characters are also identified accurately and correctly interprets the speaker-content structure.

3.1.1 Data Storage

The preprocessed collected data is stored in MongoDB. It is flexible and scalable for data retriever applications. It is a document-oriented model that allows for the storage of structured and unstructured data such as text documents, in JSON-like format. MongoDB's indexing feature gives efficient access to data.

3.1.2 Chunking

The whole conversation will be too long for to be considered as the contents. Hence the conversation is divided into smaller chunks. The first strategy while implementation was to convert into random-sized chunks.

3.1.3 Embeddings

As we want to get similar contexts to our query, we will need a medium in which we can compare our queries to contexts via vectors. So we need to convert queries and chunks of contexts into vectors using encoders. The encoder that we incorporated in our implementation is sentence-transformers.

Sentence-transformer will convert text into 768-dimensional vectors. which will look like this:-

Listing 3.1: Embdding of Hello PDEU!

```
[-6.7202e-02,  6.9565e-02, -5.6298e-02,  7.9412e-02,
   1.8159e-01,
        -6.9917e-02,  1.3635e-01, -1.8420e-02, -6.3007e-02,
            -8.2146e-02,
         5.8769e-02, -2.1332e-01, -1.3428e-02,  2.0949e-02,
              1.7970e-01,
                                                  .
                                                  .
                                                  .
                                                  .
        -3.1426e-02,  1.0189e-01, -1.9842e-01, -5.7646e-02,
            -1.9934e-01,
         7.4157e-02, -8.6848e-02,  6.6762e-03]
```

3.1.4 Indexing

Increasing the size of the data, the number of vectors in the database will reach millions. The usual way of similarity search where we will have the inner product of the query vector with each context will take more time and computational power.

Hence, to solve this problem indexing using FAISS will be used. FAISS – Facebook AI Similarity Search – is a library used for efficient similarity search. The steps for the crating index will be as follows.

- Initializing an index.
- Adding context vectors to index.
- storing the index

Listing 3.2: Implementation on Indexing

```
import faiss
```

```
d = sentence_embeddings.shape[1]
index = faiss.IndexFlatL2(d)
index.add(sentence_embeddings)
faiss.saveindex(my_index.faiss)
```

3.1.5 Similarity Search

Now, this index will be initialized, and the similarity search of each user query will be done with this example to get k nearest contexts, and these contexts will be provided to LLM with prompt as User has a query :
{User query} and contexts for the query is as :
{Contexts} Please provide an answer to the query using the given context.

3.1.6 Problem of Questions!

Once the system was tested the major issue was that most of the retrieved contexts were questions by moderators or participants instead of contexts containing actual answers. The reason for this was the encoder. As the vectors were encoded with the same encoder the questions were found similar to contexts that were questions. The solution for this problem was to incorporate a twin-tower encoder solution. i.e., two encoders, each for queries and contexts. And these are to be trained in a way that, queries (questions) get mapped to corresponding contexts (answer-containing passages).

The go-to solution for this problem was to use Dense Passage Retrieval. This was the exact solution for the problem occurring at the stage. A twin pre-trained encoder system, for questions and contexts encoding.

Listing 3.3: Context Encoding with DPR

```
from transformers import DPRContextEncoder,
    DPRContextEncoderTokenizer
tokenizer = DPRContextEncoderTokenizer.from_pretrained("
    facebook/dpr-ctx_encoder-single-nq-base")
model = DPRContextEncoder.from_pretrained("facebook/dpr-
    ctx_encoder-single-nq-base")
input_ids = tokenizer("Hello, is my dog cute ?",
    return_tensors="pt")["input_ids"]
embeddings = model(input_ids).pooler_output
```

Listing 3.4: Question Encoding with DPR

```
from transformers import DPRQuestionEncoder,
    DPRQuestionEncoderTokenizer

tokenizer = DPRQuestionEncoderTokenizer.from_pretrained(
    "facebook/dpr-question_encoder-single-nq-base")
model = DPRQuestionEncoder.from_pretrained("facebook/dpr
```

```
    -question_encoder-single-nq-base")
input_ids = tokenizer("Hello, is my dog cute ?",
    return_tensors="pt")["input_ids"]
embeddings = model(input_ids).pooler_output
```

3.1.7 Improving Chunking Strategy

The implementation of DPR gave pretty satisfying Results. However, The chunk size was either too small for some contexts or chunks were too large in some cases to be considered as context. Two changes were incorporated to resolve this.

Based on the histogram of the number of words for all conversations, A lower bound for the number of words for a chunk was decided. If a conversation has a lesser number of words than this bound then the next conversation was combined to this conversation.

Secondly, The encoder of DPR can encode a maximum of 512 tokens coming from the DPR tokenizer. So the strategy was designed that considered the first point and this and a custom function was designed.

3.2 Retrieval Augmented Generation System

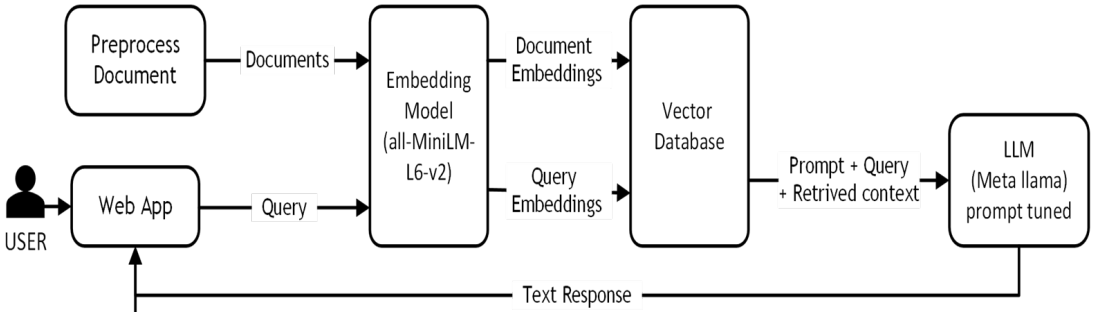


Figure 3.2: RAG system pipline

Our strategy involves building a RAG system (Retrieval Augmented Generation) that makes use of the latest Hugging Face models, as well as some advanced approaches (Bits and Bytes B optimization), to guarantee good information retrieval and generation from several PDF files.

Although the RAG design is now much easier to implement, the fundamental idea is still the same. For the input prompt, we employ a retrieval system to supplement the output produced by the LLM. With this method, we can simply expose the model to non-parametric external data, avoiding the need to retrain it on our domain-specific data and saving time on fine-tuning.

Including a conversational assistant that runs on RAG promotes a smooth and easy-to-use user interface, allowing for organic and dynamic AI conversations that raise user happiness and engagement levels. Therefore, we created a conversational system called "A REPORTER" to serve .
Four steps make up the RAG framework's answer process:

- the user poses a question.
- the system gets pertinent information from private knowledge bases.
- merges it with the question as context
- it asks the LLM to provide a response

To index our knowledge base, we first need to load the data.

Listing 3.5: PDF Loader Implementation

```
import steamlit
from llama_index.core import download_loader
from pathlib import Path
import os
PyMuPDFReader = download_loader("PyMuPDFReader")
loader = PyMuPDFReader()
documents = loader.load(file_path=Path(path.replace(os.
    sep, '/')), metadata=True)
steamlit.write(path.replace(os.sep, '/'))
```

Retrieval from PDF files presents a number of difficulties. Errors in text extraction and disorder in the row-column connections of tables within PDF files are frequent problems. Therefore, we must turn huge papers into retrievable content prior to RAG.

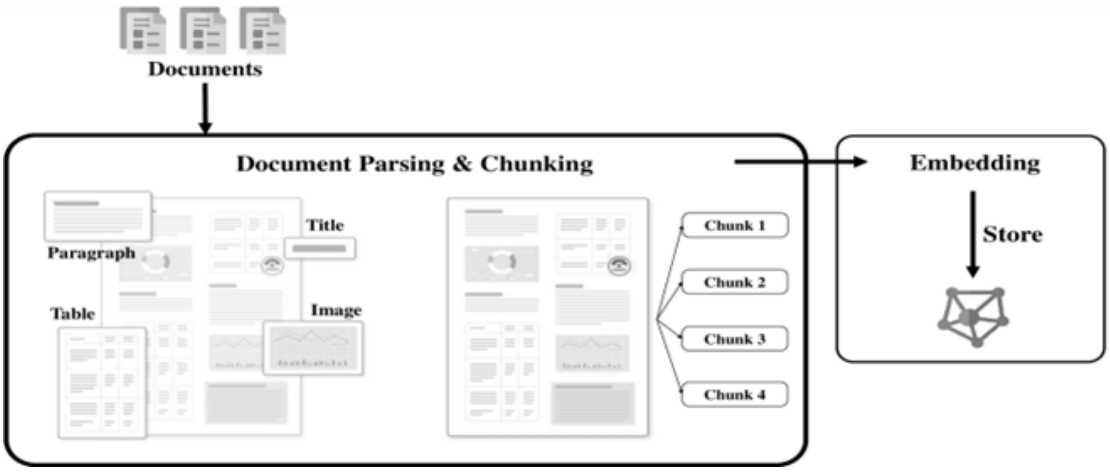


Figure 3.3: Retrieval system pipline

Document parsing and chunking is the process of removing paragraphs, tables, and other content blocks from a document and separating the removed material into manageable pieces for later retrieval.

Listing 3.6: Document parsing and chunking Implementation

```
from transformers import AutoTokenizer
tokenizer = AutoTokenizer.from_pretrained(name,
```

```
    cache_dir='/model_7b/', use_auth_token=auth_token)
```

Embedding: This converts text passages into vectors with values and saves them in a database.

Listing 3.7: Embedding Implementation

```
from langchain.embeddings.huggingface import
    HuggingFaceEmbeddings
from llama_index.embeddings.langchain import
    LangchainEmbedding
embeddings = LangchainEmbedding(HuggingFaceEmbeddings(
    model_name="all-MiniLM-L6-v2"))
```

Now we store data in vector form within the vector database.

Listing 3.8: Vector Database

```
from llama_index.core import VectorStoreIndex
index = VectorStoreIndex.from_documents(documents)
```

When we use `from_documents`, our documents are broken up into smaller pieces and parsed into Node objects, which are simple abstractions over text strings that maintain relationships and metadata.

We employed the quantization technique to get over the computational power issue. One method for lowering the accuracy of numerical numbers in a model is quantization. Quantization uses lower-precision data types, like 8-bit integers, to represent values instead of high-precision data types, like 32-bit floating-point numbers. By using this method, memory use is greatly decreased and model execution can be accelerated without sacrificing acceptable accuracy.

Figure 3.4: Quantization Example

The ability to load models in 4-bit quantization is one of this integration's primary benefits. To achieve this, call the.`from_pretrained` function with the

`load_in_4bit=True` option set. You can almost quadruple your RAM use by doing this.

Listing 3.9: Quantization and Model Implementation

```
from transformers import AutoModelForCausalLM,
    BitsAndBytesConfig
quantization_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_compute_dtype=torch.float16,
    bnb_4bit_quant_type="nf4",
    llm_int8_enable_fp32_cpu_offload=True,
    llm_int8_threshold=0.8,
    llm_int8_has_fp16_weights=True,
    bnb_4bit_use_double_quant=True,
)
model = AutoModelForCausalLM.from_pretrained(name,
        cache_dir='/model_7b/',use_auth_token=auth_token
            , torch_dtype=torch.float16,rope_scaling={"
            type": "dynamic", "factor": 2},
            quantization_config=quantization_config)
```

In the system, we ask the model to respond to our inquiry after adding some more context to the prompt. We can direct the model to adhere to the particular context we supply it to answer the question by structuring the LLM prompt as follows: an instruction, context, and question.

After employing the generic Query Engine interface with the dataset, we obtain our results along with their associated metadata.

Listing 3.10: Query Implementation

```
prompt = steamlit.text_input('Input your prompt here')
query_engine = index.as_query_engine()
steamlit.write(response.response)
steamlit.write(response.get_formatted_sources())
```

4. Results Analysis and Discussion

4.1 Results

4.1.1 PDF Scraping and OCR results

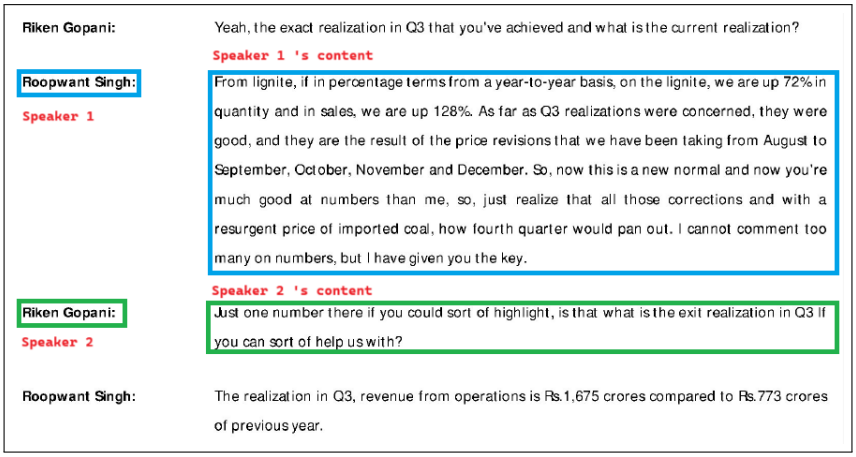


Figure 4.1: Format of conversations in meeting transcript PDF

The coloured box shows the natural reading order. Speaker's names are listed in the left column and what they said is in the right column. The reading order for the OCR tool is left to right reading in the top-to-bottom manner.

Table 4.1: Comaparative analysis of scraping and OCR library

OCR Tool	**Advantages**	**Drawbacks**
Unstructured	Better OCR compared to others.	Issue of spacing in OCR and errors in two-column format.
PyPDF2	Accurate text structure recognition.	Word formatting error.
Tesstarct OCR	Faster OCR and roboust to font type.	Not good in analyzing natural reading order, sometimes it can't recognize two columns format.
PyMuPDF	Good in natural reading order recognition and moderate OCR	Sometimes, OCR results are not very accurate.

4.1.2 Data Preprocessing Results

In the data pre-processing part meeting transcript PDFs were converted into a well-structured JSON format containing metadata of the transcripts. This metadata further can allow easy access to particular conversations for specific applications. It can work as a parameter for searching in a particular vector space. The structure of metadata is shown in Fig 4.2, Metadata contains parameters like article_id, year, month(for timestamp), and paragraph ID (para_id) for identifying the order of conversation, speaker's name and speaker's content.

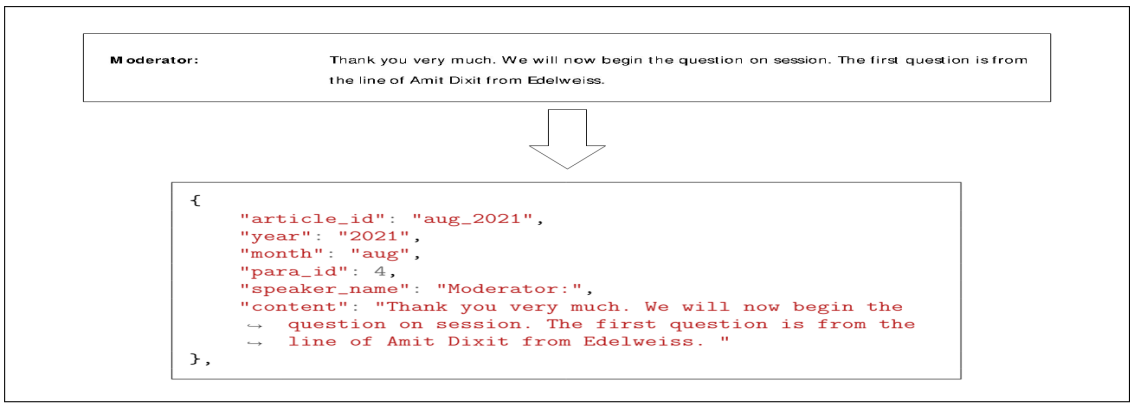


Figure 4.2: Format of conversations in meeting transcript PDF

After preprocessing, the file is stored in the database, for easy and fast access from anywhere.

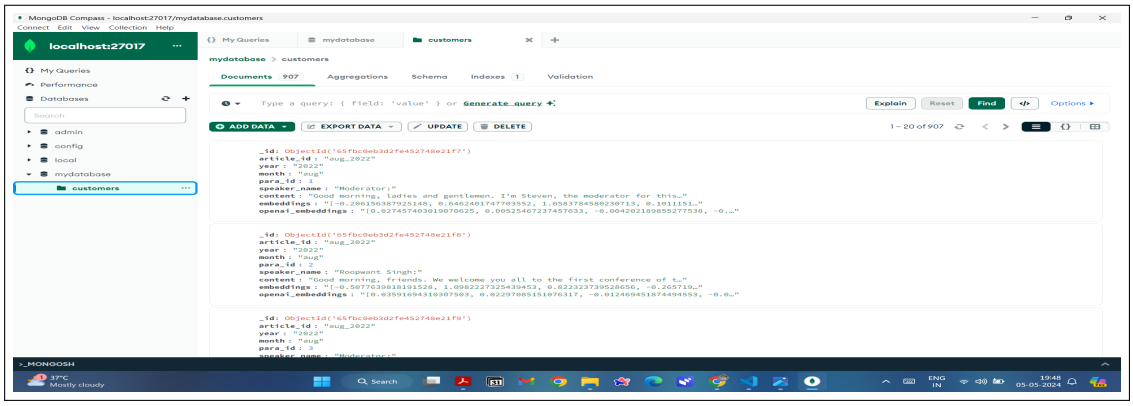


Figure 4.3: MongoDB Compass

DPR will retrieve K = 10 conversations that have a higher similarity score with the question asked and we summarize that information using LLMs, LLMs provide a better understanding of conversational data.

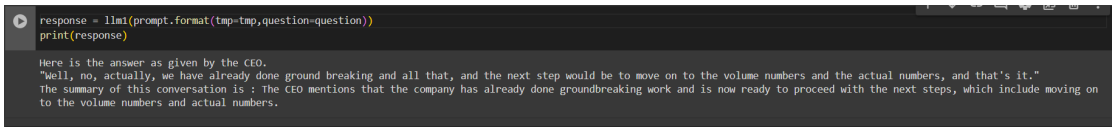


Figure 4.4: LLAMA DPR Summary

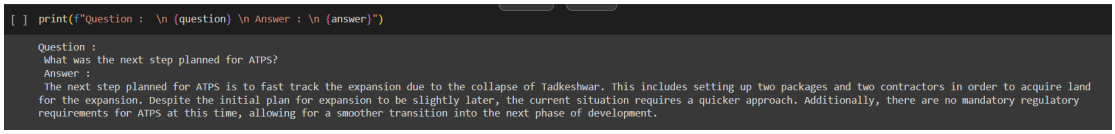


Figure 4.5: ChatGPT DPR Summary

For comparative analysis, meta LLAMA and OpenAI's ChatGPT LLMs are used. ChatGPT outperformed LLAMA 7B model, it gives a better understanding

to retrieve data for further understanding with a great summarization. ChatGPT have a higher knowledge understanding compared to LLAMA which helps to understand and summarize a complex conversation.

4.1.3 LLAMA2 RAG Chatbot results

Using the streamlit Python library chatbot interface is created. User Interface allows a better conversation experience with LLAMA2 LLM. Also, previous chat sessions can be stored in temporary chat history, this is done using Langchain. It allows LLM to have some previous context over a particular query. A query can be explained to LLM using a chat interface.

For LLAMA RAG two llama models are used:

- Llama2-7B

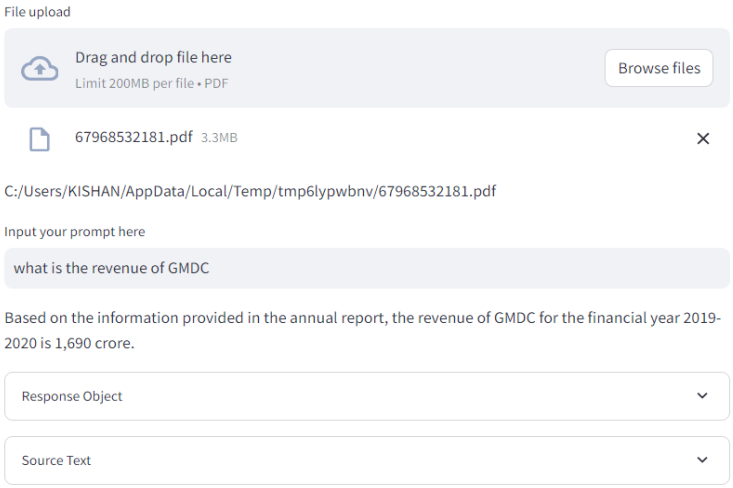


Figure 4.6: Llama2-7B Model for table understanding task

- Llama2-13B

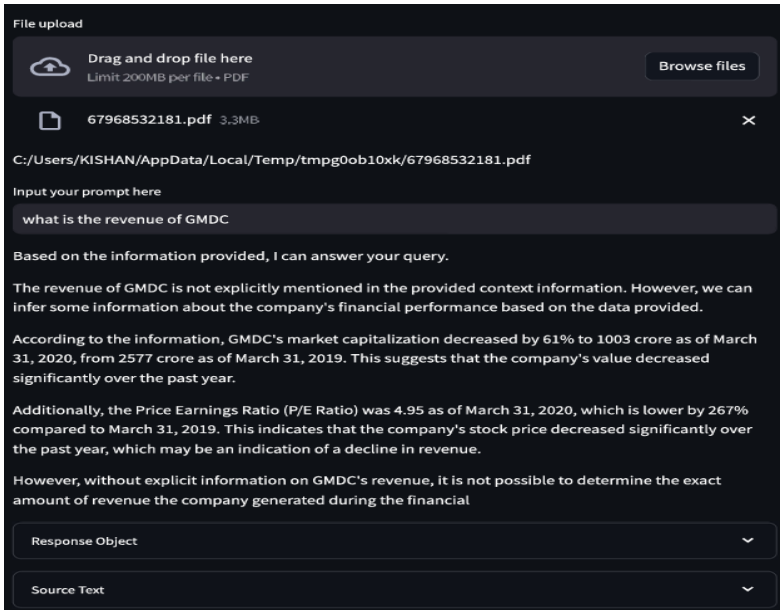
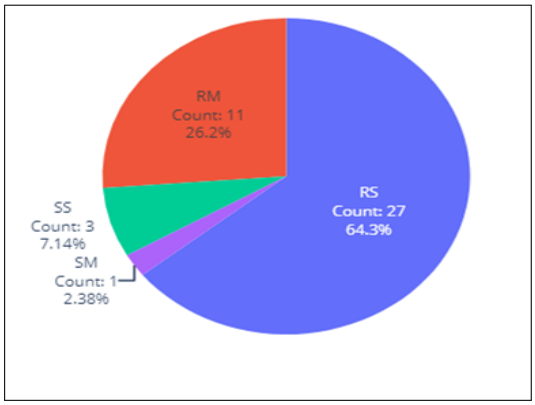
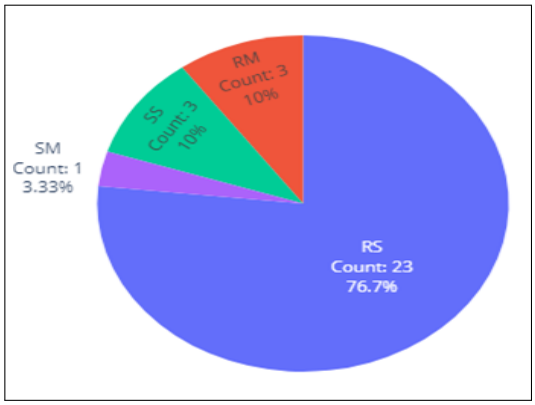


Figure 4.7: Llama2-13B Model for table understanding task

7b Model is one of the fastest because of less parameters compared to other models. Sometimes it gives clear and concise answers, whereas the 13b model gives a better explainable answer but it takes longer time than the 7b model on a local device running on Geforce RTX3050Ti 6GB GPU.



(a) Categories in Actual Dataset



(b) Categories used for Validation

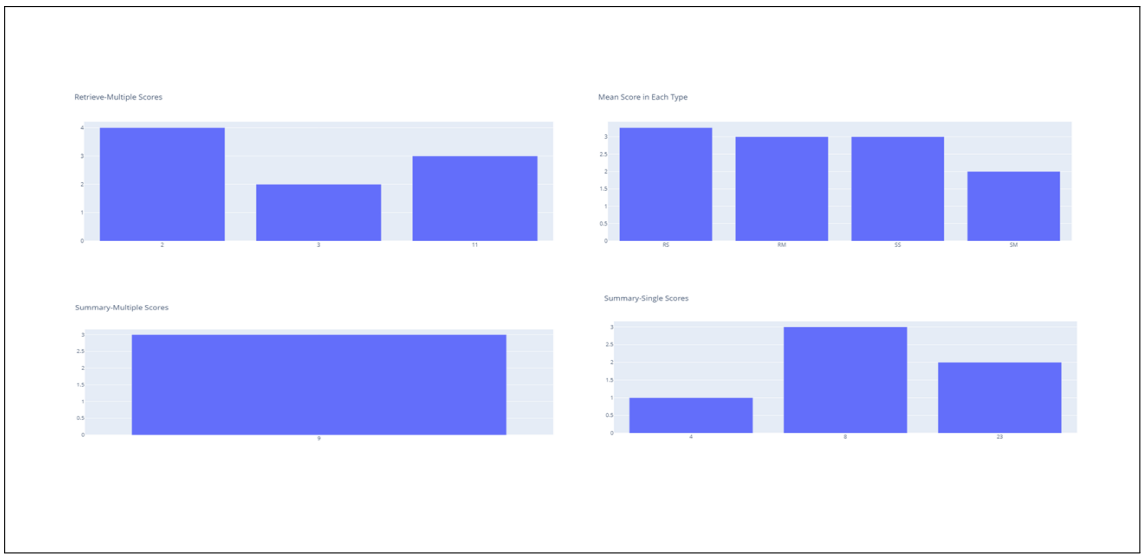


Figure 4.9: Bar chart of Scores Mean, RM, SM, SS

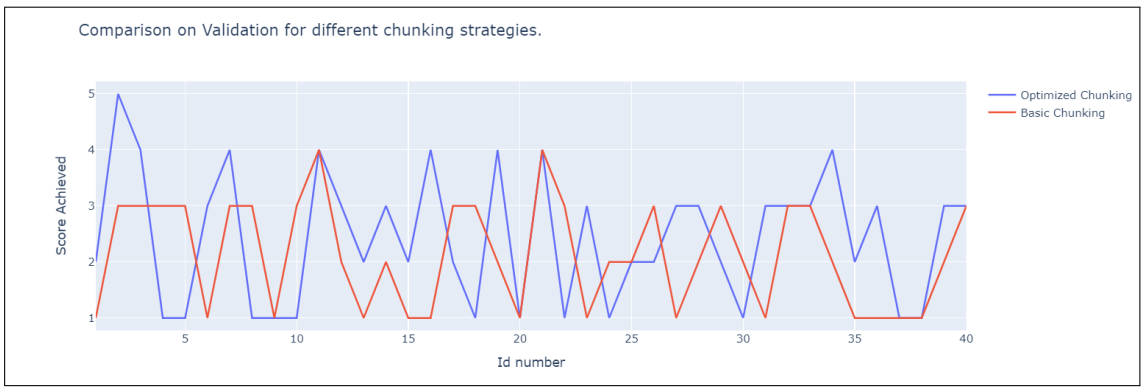


Figure 4.10: Comparison of different chunking strategies

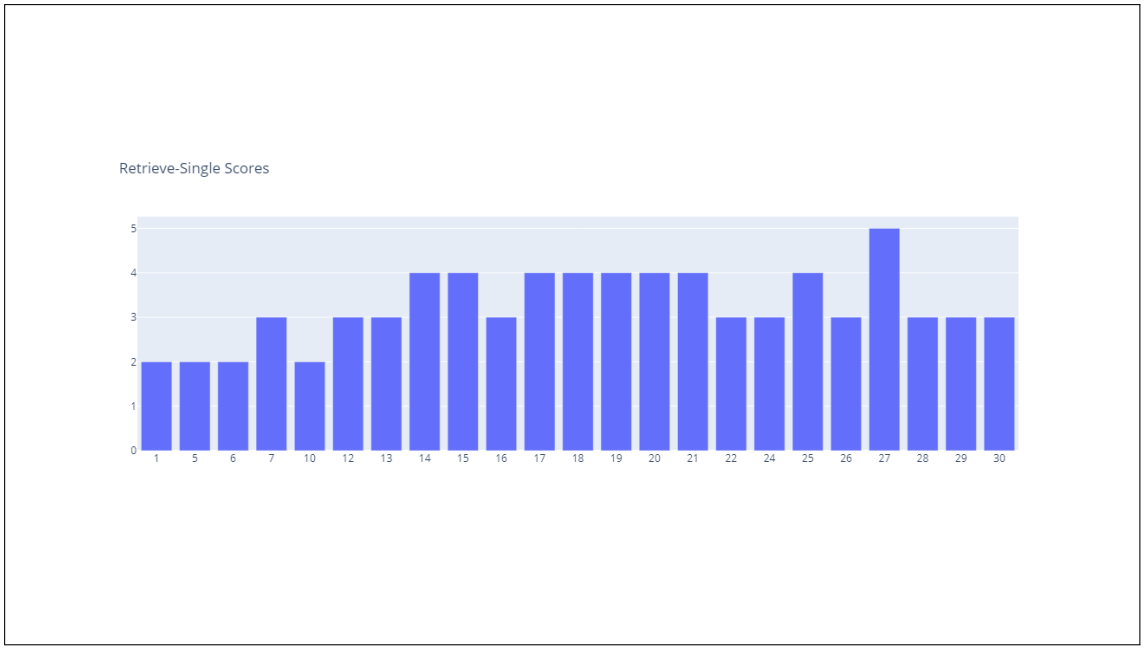


Figure 4.11: Bar chart of Scores of RS

5. Conclusion

In order to improve concall talks in corporate settings, we have looked into the use of retrieval augmented generation, or RAG, in this thesis. Through the integration of cutting-edge natural language processing methods with cutting-edge language models like Llama2-7b and Llama2-13b, we have created a novel method for speaker diarization that makes use of both generative and retrieval-based capabilities. In reliably identifying speakers, summarizing discussions, and enabling information retrieval during conference calls, our methodology has shown encouraging results.

We have demonstrated the efficacy of RAG in tackling the problems associated with business communication, such as context retention, speaker attribution, and information overload, through thorough testing and assessment. We have significantly improved conference call content comprehension, organization, and accessibility by utilizing large-scale language models and linking them with retrieval methods. The information extraction capabilities of our RAG model using Llama2-7b and Llama2-13b are impressive. However, there seem to be occasional calculation errors arising after processing certain tabular data.

6. Scope for Future Work

Retrieval-augmented generation (RAG) for conference calls in business environments has numerous intriguing research and development directions ahead of it:

1. Better Contextual Understanding: In order to produce more pertinent and correct solutions, it is imperative to improve the model's capacity to comprehend and interpret business talks. Subsequent investigations may concentrate on enhancing contextual embeddings, integrating knowledge graphs particular to a given domain, and creating more effective systems for encapsulating the subtleties of business discussions.

2. Personalized Interaction: You can increase conference call productivity and engagement by customizing responses to each participant's needs and preferences. Prospective RAG systems may utilize participant profiles, past interactions, and instantaneous feedback to customize generated content, so offering more focused and efficient communication assistance.

3. Dynamic Content summarizing: You can increase information retention and comprehension by creating dynamic summarizing strategies that change with the content and structure of conference calls. Subsequent RAG models could include real-time content analysis, user preferences, and relevance feedback to produce succinct and useful summaries that are customized to each participant's unique requirements.

4. Intelligent Assistants: By transforming RAG systems into intelligent virtual assistants, which can handle agenda management, action item tracking, and meeting scheduling, among other conference call functions, communication procedures can be streamlined and teamwork improved. Conference calls can become more effective and productive exchanges by including natural language understanding, task automation, and proactive support features.

5. Multimodal Integration: Enhancing the context and richness of engagement in conference calls can be achieved by integrating many modalities, such as audio, video, text, and visual signals. In the future, RAG systems might make use of sophisticated multimodal fusion techniques to merge data from several sources, making interaction analysis, content summarizing, and speaker diarization more thorough.

7. Appendix

Steamlit:

Steamlit offers a user-friendly web interface that makes it easy for anyone, anywhere, to demo your machine-learning model quickly and efficiently! These are its principal aspects:

- **Quick and simple setup** : Pip can be used to install Steamlit. It just takes a few lines of code to add a Steamlit interface to your project. Utilize any Python library on your PC with ease. Steamlit can run any Python function that you can write.
- **Display and distribute** : Steamlit may be shown as a webpage or integrated into Python notebooks. When a Steamlit interface is used, it can immediately create a public link that you may share with others so they can use their own devices to interact remotely with the model on your computer.
- **Permanent Hosting** : Once an interface has been developed on Hugging Face, it can be hosted indefinitely, ensuring sustained accessibility and usability for users.

Bibliography

[1] E. M. Voorhees *et al.*, "The trec-8 question answering track report.," in *Trec*, vol. 99, pp. 77–82, 1999.

[2] D. Moldovan, M. Paşca, S. Harabagiu, and M. Surdeanu, "Performance issues and error analysis in an open-domain question answering system," *ACM Transactions on Information Systems (TOIS)*, vol. 21, no. 2, pp. 133–154, 2003.

[3] D. Chen, A. Fisch, J. Weston, and A. Bordes, "Reading wikipedia to answer open-domain questions," *arXiv preprint arXiv:1704.00051*, 2017.

[4] S. Robertson, H. Zaragoza, *et al.*, "The probabilistic relevance framework: Bm25 and beyond," *Foundations and Trends® in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[5] A. Shrivastava and P. Li, "Asymmetric lsh (alsh) for sublinear time maximum inner product search (mips)," *Advances in neural information processing systems*, vol. 27, 2014.

[6] K. Lee, M.-W. Chang, and K. Toutanova, "Latent retrieval for weakly supervised open domain question answering," *arXiv preprint arXiv:1906.00300*, 2019.

[7] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2018.

[8] J. Bromley, I. Guyon, Y. LeCun, E. Säckinger, and R. Shah, "Signature verification using a" siamese" time delay neural network," *Advances in neural information processing systems*, vol. 6, 1993.

[9] T. Kwiatkowski, J. Palomaki, O. Redfield, M. Collins, A. Parikh, C. Alberti, D. Epstein, I. Polosukhin, J. Devlin, K. Lee, *et al.*, "Natural questions: a benchmark for question answering research," *Transactions of the Association for Computational Linguistics*, vol. 7, pp. 453–466, 2019.

[10] S. Mussmann and S. Ermon, "Learning and inference via maximum inner product search," in *International Conference on Machine Learning*, pp. 2587–2596, PMLR, 2016.

[11] T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado, and J. Dean, "Distributed representations of words and phrases and their compositionality," *Advances in neural information processing systems*, vol. 26, 2013.

[12] J. Pennington, R. Socher, and C. D. Manning, "Glove: Global vectors for word representation," in *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*, pp. 1532–1543, 2014.

[13] A. Conneau, D. Kiela, H. Schwenk, L. Barrault, and A. Bordes, "Supervised learning of universal sentence representations from natural language inference data," *arXiv preprint arXiv:1705.02364*, 2017.

[14] V. Ashish, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, p. I, 2017.

[15] M. Henderson, R. Al-Rfou, B. Strope, Y.-H. Sung, L. Lukács, R. Guo, S. Kumar, B. Miklos, and R. Kurzweil, "Efficient natural language response suggestion for smart reply," *arXiv preprint arXiv:1705.00652*, 2017.

[16] M. Iyyer, V. Manjunatha, J. Boyd-Graber, and H. Daumé III, "Deep unordered composition rivals syntactic methods for text classification," in *Proceedings of the 53rd annual meeting of the association for computational linguistics and the 7th international joint conference on natural language processing (volume 1: Long papers)*, pp. 1681–1691, 2015.

[17] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[18] D. Cer, M. Diab, E. Agirre, I. Lopez-Gazpio, and L. Specia, "Semeval-2017 task 1: Semantic textual similarity-multilingual and cross-lingual focused evaluation," *arXiv preprint arXiv:1708.00055*, 2017.

[19] B. Kulis *et al.*, "Metric learning: A survey," *Foundations and Trends® in Machine Learning*, vol. 5, no. 4, pp. 287–364, 2013.

[20] W.-t. Yih, K. Toutanova, J. C. Platt, and C. Meek, "Learning discriminative projections for text similarity measures," in *Proceedings of the fifteenth conference on computational natural language learning*, pp. 247–256, 2011.

[21] D. Gillick, S. Kulkarni, L. Lansing, A. Presta, J. Baldridge, E. Ie, and D. Garcia-Olano, "Learning dense representations for entity retrieval," *arXiv preprint arXiv:1909.10506*, 2019.

[22] S. Min, D. Chen, L. Zettlemoyer, and H. Hajishirzi, "Knowledge guided text retrieval and reading for open domain question answering," *arXiv preprint arXiv:1911.03868*, 2019.

[23] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, *et al.*, "Llama 2: Open foundation and fine-tuned chat models," *arXiv preprint arXiv:2307.09288*, 2023.